

ΑΣΦΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

Καράς Άγγελος

ΤΙΤΛΟΣ ΕΡΓΑΣΙΑΣ: RSA και Ψηφιακές Υπογραφές.

➤ Δημιουργία ιδιωτικού κλειδιού:

1. Έστω ο τύπος $e * d = 1 \bmod (p-1) * (q-1)$ για την δημιουργία ενός RSA κλειδιού και q, p γνωστά. Θα πρέπει να φτιάξουμε δύο μεταβλητές tmp και $tmp1$:

```
BIGNUM *tmp1 = BN_new();  
BIGNUM *tmp2 = BN_new();
```

```
BN_sub(tmp1, p, one);  
BN_sub(tmp2, q, one);
```

- Έπειτα για να ολοκληρώσουμε το δεύτερο σκέλος της σχέσης αρκεί να :
`BN_mul(formod, tmp1, tmp2, ctx);`
- Τέλος με `BN_mod_inverse(dec_key, e, formod, ctx);` καταλήγουμε στην αρχική μας σχέση.

```
1 /* askhsh.c */  
2 #include <stdio.h>  
3 #include <openssl/bn.h>  
4  
5 #define NBITS 256  
6  
7 void printBN(char *msg, BIGNUM *a)  
8 {  
9     char *number_str = BN_bn2hex(a);  
10    printf("%s %s\n", msg, number_str);  
11    OPENSSL_free(number_str);  
12 }  
13 void main () {  
14  
15     //ΑΡΧΙΚΟΠΟΙΟΥΜΕ ΤΙΣ ΤΙΜΕΣ p, q, n, e, one...  
16     BN_CTX *ctx = BN_CTX_new();  
17     BIGNUM *p = BN_new();  
18     BIGNUM *q = BN_new();  
19     BIGNUM *n = BN_new();  
20     BIGNUM *e = BN_new();  
21     BIGNUM *one = BN_new();  
22     BIGNUM *tmp1 = BN_new();  
23     BIGNUM *tmp2 = BN_new();  
24     BIGNUM *formod = BN_new(); // formod = tmp1 * tmp2  
25     BIGNUM *dec_key = BN_new();  
26  
27     BN_hex2bn(&p, "953AAB9B3F23ED593FBDC690CA10E703");  
28     BN_hex2bn(&q, "C34EFC7C4C2369164E953553CDF94945");  
29     BN_hex2bn(&e, "0D88C3");  
30     BN_hex2bn(&one, "1");  
31  
32     BN_mul(n, q, p, ctx); // n = q * p  
33     BN_sub(tmp1, p, one); //tmp1 = p - 1  
34     BN_sub(tmp2, q, one); //tmp2 = q - 1  
35     BN_mul(formod, tmp1, tmp2, ctx); // formod = tmp1 * tmp2  
36     BN_mod_inverse(dec_key, e, formod, ctx);  
37     printf("Private Key \n");  
38     printBN("dec_key : ", dec_key);  
39     printBN("n : ", n);  
40     printf("-----\n");  
41     printf("Public key \n");  
42     printBN("e:", e);  
43  
44     //return 0;  
45 }
```

➤ Κρυπτογράφηση μηνύματος

1. Δίνεται το n , e και το d απο την άσκηση 1.

- Δημιουργούμε ένα κρυπτογραφημένο μήνυμα με την εντολή , για δεκαεξαδική μορφή:

```
python -c 'print("Aggelos Karras").encode("hex")'
```

```
aggelos@aggelos-All-Series:~$ python -c 'print("Aggelos Karras").encode("hex")'
416767656c6f73204b6172726173
```

- Στη συνέχεια για να κρυπτογραφήσουμε το μήνυμα θα πρέπει να βάλουμε την εντολή :

```
BN_mod_exp(enc, name, e, n, ctx);
```

◆ Όπου name το δεκαεξαδικό μήνυμα απο την εντολή της python.

```
/* askhsh.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM *a)
{
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

void main () {

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *name= BN_new();
    BIGNUM *enc = BN_new();
    BIGNUM *dec = BN_new();
    BIGNUM *dec_key = BN_new();

    BN_hex2bn(&name, "416767656c6f73204b6172726173");
    BN_hex2bn(&dec_key, "63F67E805D8DEB0B4182C57C3DC24F3C1350CF182E8ABF85FD24062A3BC7F2EB");
    BN_hex2bn(&n, "71D9BBC5C01F9B50DDFE5F2EC331FAB21081009D014E9615C277670C61591ECF");
    BN_hex2bn(&e, "0D88C3");

    BN_mod_exp(enc, name, e, n, ctx);
    BN_mod_exp(dec, enc, dec_key, n, ctx);

    printBN("Krimeno Onoma : ", name);
    printBN("Kriptografimeno mhnuma : ", enc);
    printBN("Apokriptografimeno mhnuma : ", dec);
}
```

```
[03/30/21]seed@VM:~$ ./askhsh2
Krimeno Onoma : 416767656C6F73204B6172726173
Kriptografimeno mhnuma : 056C1613638A34524517722AC0AF536D0EAD32C7F142839913A5DA
69D6562D34
Apokriptografimeno mhnuma : 416767656C6F73204B6172726173
[03/30/21]seed@VM:~$
```

➤ Αποκρυπτογράφηση μηνύματος

1. Δίνεται η, e, d και ένα μήνυμα C το οποίο στο πρόγραμμα που ακολουθεί παίρνει την τιμή enc.

```
/* askhsh.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM *a)
{
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

void main () {

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *name= BN_new();
    BIGNUM *enc = BN_new();
    BIGNUM *dec = BN_new();
    BIGNUM *dec_key = BN_new();

    BN_hex2bn(&name, "416767656c6f73204b6172726173");
    BN_hex2bn(&dec_key, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");
    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&enc, "B3AF0A70793BB53492B5311AED5EA843D94661924C97A446E9DD75846DF860DF");
    BN_hex2bn(&e, "010001");

    BN_mod_exp(dec, enc, dec_key, n, ctx);

    printBN("Krimeno Onoma : ", name);
    printBN("Kriptografimeno mhnuma : ", enc);
    printBN("Apokriptografimeno mhnuma : ", dec);
}
```

```
[03/30/21]seed@VM:~$ ./askhsh3
Krimeno Onoma : 416767656C6F73204B6172726173
Kriptografimeno mhnuma : B3AF0A70793BB53492B5311AED5EA843D94661924C97A446E9DD75
846DF860DF
Apokriptografimeno mhnuma : 494345205365637572697479206C616220323032302D3231
[03/30/21]seed@VM:~$
```

➤ Υπογραφή μηνύματος

python -c 'print("Aggelos Karras".encode("hex"))' στο πρόγραμμα έχει την τιμή signed_mes
python -c 'print("Agkelos Karras".encode("hex"))' στο πρόγραμμα έχει την τιμή signed_mes2

Η τιμή του hex θα αλλάξει γιατί οι τιμές αυτές είναι από τον πίνακα ASCII . Οπότε λογικό να αλλάξουν οι τιμές.

```
/* askhsh.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM *a)
{
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main () {

    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *enc = BN_new();
    BIGNUM *dec = BN_new();
    BIGNUM *dec_key = BN_new();
    BIGNUM *signed_mes = BN_new();
    BIGNUM *signed_mes2 = BN_new();
    BIGNUM *mes = BN_new();
    BIGNUM *mes2 = BN_new();

    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&dec_key, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&mes, "416767656c6f73");
    BN_hex2bn(&mes2, "41676b656c6f73");

    BN_mod_exp(signed_mes, mes, dec_key, n, ctx);
    printBN("Test 1: ", signed_mes);

    BN_mod_exp(signed_mes2, mes2, dec_key, n, ctx);
    printBN("Test 2: ", signed_mes2);

    return 0;
}
```

```
[03/30/21]seed@VM:~$ ./askhsh4
Test 1: 3B31A4CC0FD81D1B99C72FF1E652269B4C8EE03EEDE8B2FB03E1B6A7B87FD473
Test 2 : 6EE66029EF4BEBDF2572F2B351BAB812DDD7E24676026827018D85035B8E723E
[03/30/21]seed@VM:~$
```


➤ Επαλήθευση υπογραφής

1. Δίνεται S, n, e .

- Με την εντολή BN_mod_exp(msg, S, e, n, ctx); ελέγχουμε αν το μήνυμα είναι σωστό .

```
/* askhsh.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM *a)
{
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(int argc, char const *argv[])
{
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *S = BN_new();
    BIGNUM *S1 = BN_new();
    BIGNUM *msg = BN_new();
    BIGNUM *msg1 = BN_new();

    BN_hex2bn(&n, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F");
    BN_hex2bn(&S1, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F");

    BN_mod_exp(msg, S, e, n, ctx);
    printBN("Message: ", msg);

    BN_mod_exp(msg1, S1, e, n, ctx);
    printBN("Wrong Message: ", msg1);

    return 0;
}
```

2. Στη δεύτερη περίπτωση κάνουμε το ανάλοδο.

```
/* askhsh.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM *a)
{
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(int argc, char const *argv[])
{
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *S = BN_new();
    BIGNUM *verify_msg = BN_new();
    BIGNUM *msg = BN_new();
    BIGNUM *check = BN_new();

    BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
    BN_hex2bn(&e, "010001");
    BN_hex2bn(&S, "DB3F7CDB93483FC1E70E4EACA650E3C6505A3E5F49EA6EDF3E95E9A7C6C7A320");
    BN_hex2bn(&verify_msg, "506c65617365207472616e73666572206d652024323030302e416c6963652e");

    BN_mod_exp(check, S, e, n, ctx);

    if ( !BN_cmp(verify_msg, check) ) {
        printf("To mhnuma einai swsto\n");
    }
    else {
        printf("Ti mhnuma den einai swsto\n");
    }

    return 0;
}
```

➤ Μη αυτόματη επαλήθευση πιστοποιητικού x.509.

Με την εντολή openssl s_client -connect (link που θέλουμε):443 -showcerts μπορούμε να κατεβάσουμε το πιστοποιητικό από έναν πραγματικό web server . Η πρώτη υπογραφή είναι του host και το δεύτερο κλειδί του client .

[illegible]

- Από αυτήν πρώτη υπογραφή μπορούμε να πάρουμε το n με την εντολή .
openssl x509 -in c1.pem -noout -modulus >> n.txt
- Ενώ για να μπορούμε να πάρουμε το e .

- openssl x509 -in c1.pem -text -noout
- Ψάχνοντας το output της εντολής παρατηρούμε `Exponent: 65537 (0x10001)`
- Από την εντολή openssl x509 -in c0.pem -text -noout εκτυπώνουμε όλα τα πεδία . Στη συνέχεια αντιγράφουμε όλο το Signature Algorithm σε ένα sign.txt και με την εντολή
`cat sign.txt | tr -d '[:space:]' >> s.txt` αποκτούμε το δημόσιο κλειδί S .
- Στη συνέχεια για να πάρουμε το hash.txt αρκεί να τρέξουμε την εντολή
`openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout`
και στην συνέχεια `sha256sum c0_body.bin >> hash.txt`

◆ Η επαλήθευση:

```
/* askhsh.c */
#include <stdio.h>
#include <openssl/bn.h>

#define NBITS 256

void printBN(char *msg, BIGNUM *a)
{
    char *number_str = BN_bn2hex(a);
    printf("%s %s\n", msg, number_str);
    OPENSSL_free(number_str);
}

int main(int argc, char const *argv[])
{
    BN_CTX *ctx = BN_CTX_new();
    BIGNUM *e = BN_new();
    BIGNUM *n = BN_new();
    BIGNUM *S = BN_new();
    BIGNUM *verify_msg = BN_new();
    BIGNUM *msg = BN_new();
    BIGNUM *check = BN_new();
    BIGNUM *hash = BN_new();

    BN_hex2bn(&n,
"D67333D6D73C20D000D21745B8D63E07A23FC741EE3230C9B06CFDF49FCB12980F2D3F8D4D010C820F177F622EE9B
BN_hex2bn(&e, "010001");
    BN_hex2bn(&S,
"8b606e0da450439a9f5dcdbea64f7a802daca30925e17c8dbb8cc6c9dd3be511f3e279e81b3684513b2d6f13886c3
    BN_hex2bn(&verify_msg, "506c65617365207472616e73666572206d652024323030302e416c6963652e");
    BN_hex2bn(&hash, "238ace9ab519e70a6a286c9f6cb63ca01f71400985833c8b3e1e14ee5dc3437f");
    BN_mod_exp(check, S, e, n, ctx);

    printf("To mhnoma einai swsto\n" check);
}

return 0;
}
```

- Τέλος με το αρχείο elegxos.py ελέγχουμε αν το hash.txt είναι ίδιο με το out.txt το οποίο το πήραμε από τον κώδικα της άσκησης 6 .

```
aggelos@aggelos-All-Series:~/Εγγραφα/IT Security/Εργαστήριο/askhsh 6$ python elegxos.py  
correct signature
```

- ◆ Το αρχείο elegxos.py περιέχει :

```
with open('out.txt') as f:  
    with open('hash.txt') as h:  
        if h.read().upper() in f.read():  
            print("correct signature")
```